

SOR



Marco Tessari



Giovanni Zanella



Giovanni Peruzzi



Sebastiano Priante

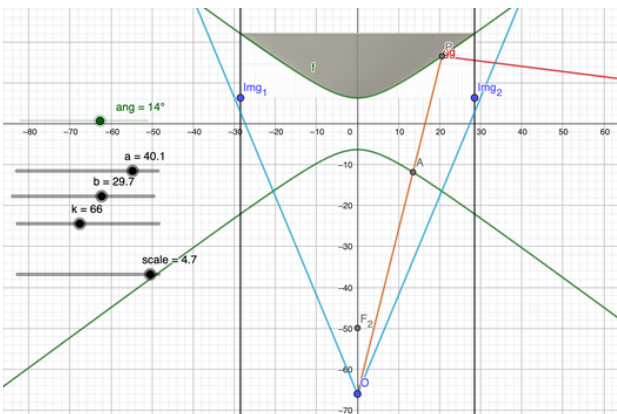


Alessandro Falcari

RoboCup Junior Soccer 2026 Vision league

SPECCHIO IPERBOLICO

Realizzato a partire da una barra tonda in acciaio inossidabile da 60 mm, il nostro specchio iperbolico è stato tornito e lucidato con precisione fino a ottenere una finitura a specchio. Questo componente ottico personalizzato offre un campo visivo a 360° ottimizzato per ridurre al minimo la distorsione, essenziale per una completa consapevolezza della situazione durante la partita.



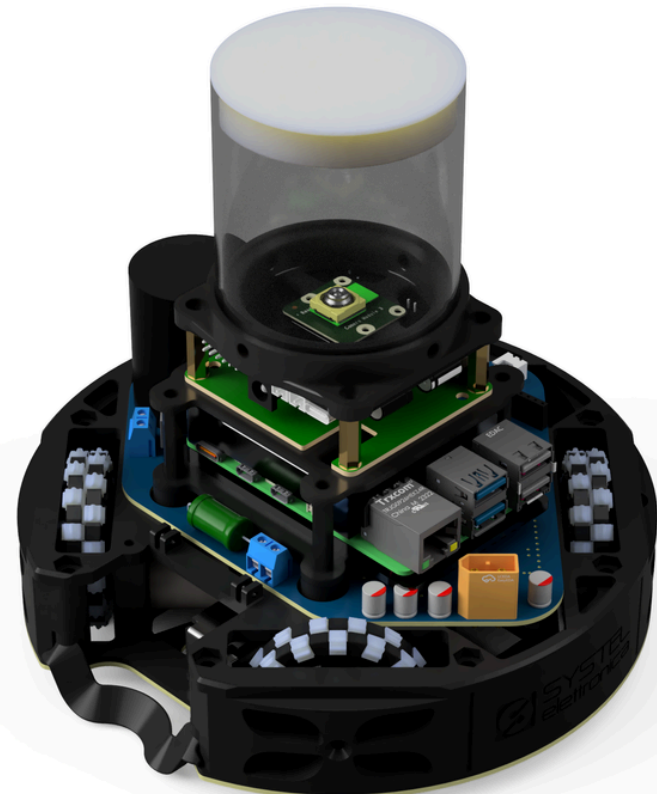
MECHANICAL

- Chassis: Design ultra-compatto con baricentro ottimizzato per la stabilità.
- Motorizzazione: Pololu 25D High-Power con encoder integrato.
- Trasmissione: Rapporto di riduzione 6:1 per un output di coppia superiore.
- Meccanica di Protezione: Architettura a motori traslati per il totale assorbimento dello stress radiale, aumentando la vita utile dei componenti e la precisione del movimento.
- Performance: Accelerazione reattiva e stabilità torsionale migliorata in fase di dribbling.



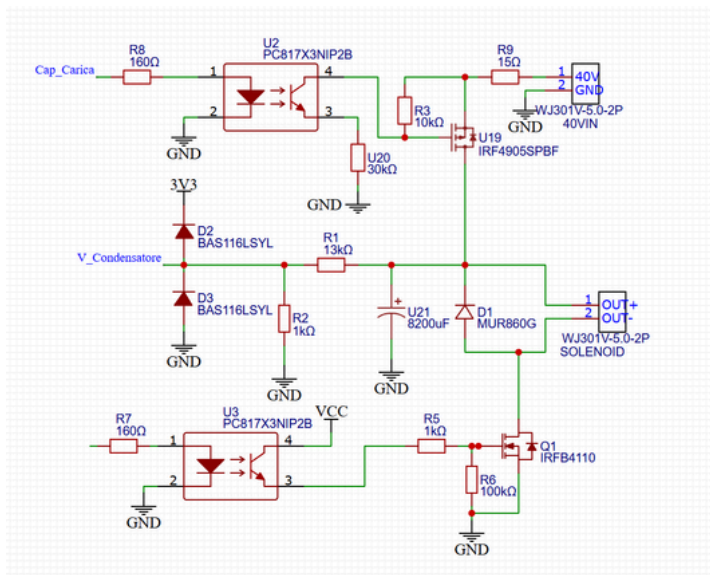
VISION

Grazie alla telecamera Raspberry Pi Camera Module 3, sfruttando la libreria Python picamera2, gestiamo distanza, colori e altri dati che poi il Raspberry Pi analizza e cerca la migliore soluzione per il movimento. L'immagine catturata viene elaborata in tempo reale attraverso una pipeline di computer vision basata su OpenCV. Il robot utilizza uno specchio iperbolico omnidirezionale che offre un campo visivo a 360°: l'immagine circolare viene "srotolata" in una striscia panoramica tramite trasformazione polare, fornendo al sistema una consapevolezza costante dell'intero campo di gioco. Vedi SPECCHIO IPERBOLICO



KICKER

Abbiamo progettato un circuito di tiro custom che utilizza un condensatore caricato a 40V per azionare un solenoide CB0830. La logica di commutazione è affidata a due MOSFET: un tipo P per la ricarica e un tipo N per lo sparo. Un elemento distintivo del nostro design è l'uso di optocouplers interfacciati con l'MCU. Questo approccio protegge il Raspberry Pi Pico separando fisicamente i circuiti di potenza da quelli di segnale. Il risultato è un sistema di tiro estremamente reattivo, capace di lanciare la palla con una forza costante e in totale sicurezza per l'elettronica di bordo.



SOFTWARE

Elenco di software utilizzati:

- Robot Behavior: Unity, Unity ML-Agents, PyTorch, Python, ONNX, HailoRT
- Computer Vision: OpenCV, picamera2
- Varie: Flask, AnyDesk, MicroPython, VSC
- Modellazione: Fusion

ELETTRONICA

Abbiamo ingegnerizzato due schede elettroniche dedicate per massimizzare l'efficienza del robot. La prima scheda è un array di 12 sensori reflectance posizionati strategicamente per monitorare il perimetro di gioco con alta precisione. La seconda scheda gestisce l'interfaccia di comunicazione tra la "mente" del robot (Raspberry Pi 5) e il microcontrollore (Raspberry Pi Pico). Questa configurazione permette al Pi 5 di elaborare i dati della camera a 360°, inviando comandi rapidi al Pico, che si occupa del controllo dei motori e della lettura dei sensori di linea. L'uso di PCB custom ci ha permesso di ridurre i cablaggi, minimizzando i rischi di disconnessione e interferenze elettromagnetiche.

STRATEGY

Il comportamento del nostro robot attaccante è interamente guidato da una rete neurale addestrata in una simulazione Unity tramite ML-Agents, un framework di reinforcement learning. Nella simulazione, due agenti si sfidano in una partita di calcio 1v1 imparando in modo autonomo per tentativi ed errori. L'addestramento utilizza PPO (Proximal Policy Optimization) con una strategia di curriculum learning. La rete addestrata viene esportata come modello ONNX e caricata direttamente sul Raspberry Pi 5 del robot, dove viene eseguita in tempo reale, grazie alla piattaforma Hailo (AI Hat).

Così il nostro robot non è più vincolato a rigide condizioni logiche!

Il robot difensione invece prevede la traiettoria della palla e cerca di pararla spostandosi in parallelo alla porta. Spinnet fulco dell'allenamento IA, che guida l'allenamento con una ricompensa proporzionale alla qualità del tiro:

```
protected override void OnHitBall()
{
    // 1. Calcolo Forza
    float force = ballBehavior.rb.velocity.magnitude;
    force = Mathf.Clamp(force, 0f, 1.6f);
    float val = (0.1f + Mathf.Pow(force + 2f, 1.2f)*0.4f) * 10f;

    // 2. Calcolo della precisione (Verso la porta)
    Vector3 direzionePalla = ballBehavior.rb.velocity.normalized;
    Vector3 direzionePorta = (myGoalToScore().transform.position -
        ballBehavior.transform.position).normalized;

    // Il Dot Product restituisce 1 se le direzioni sono identiche, 0 se sono perpendicolari
    float precisione = Vector3.Dot(direzionePalla, direzionePorta);

    // 3. Normalizziamo la precisione (ignoriamo i tiri all'indietro)
    float precisionePremio = Mathf.Max(0, 0.6f + precisione * 1.2f);

    // 4. Calcolo Finale: Moltiplichiamo la forza per la precisione
    // In questo modo, un tiro fortissimo ma nella direzione sbagliata vale poco.
    val = RewardConf.hitBallForceAlpha;
    precisionePremio = RewardConf.hitBallDirectionAlpha;

    float rewardFinale = Math.Clamp(val * precisionePremio, 0f, 4.5f +
        RewardConf.hitBallDirectionAlpha * RewardConf.hitBallForceAlpha);

    AddRewardContestantAgent(rewardFinale, "hit ball toward goal");

    gameManager.AgentHitBall(this);
}
```

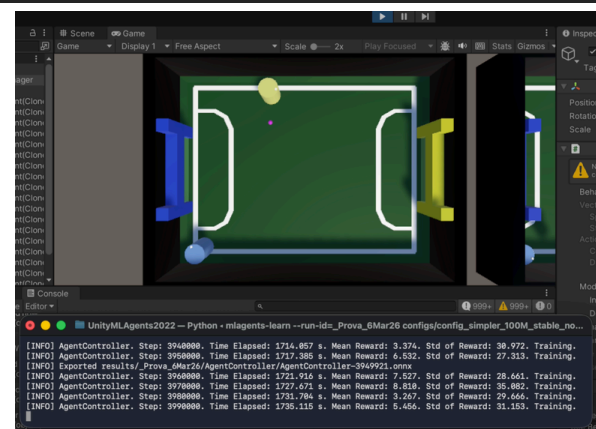


Immagine dell'allenamento degli agenti su Unity

SPONSOR

